CLASE MAGISTRAL · ARQUITECTURA IA

Orquestación de Memoria en Agentes

«La memoria la construye tu aplicación, no el LLM.»

Contexto Dinámico

Estado Persistente



01 · ARQUITECTURA BASE

El LLM es el Actor. Tu Aplicación es el Director.

El LLM (El Actor)

Brillante en interpretación instantánea, pero completamente desmemoriado al bajar el telón.

Stateless (Sin memoria)

La Aplicación (El Director)

Sostiene el libreto. Registra la escena previa y le susurra al actor el contexto exacto antes de cada toma.

Stateful (Orquestador de Estado)

02 · EL RETO DEL CONTEXTO

¿Por qué un Agente colapsa en el Turno 2?

TURNO 1 ✓

«Hola, me llamo Alex y programo en Python.»

El LLM responde amablemente y finaliza la petición HTTP.

TURNO 2 (AMNESIA) ✗

«¿Qué lenguaje te dije que uso?»

«No me has mencionado ningún lenguaje aún.»

La Solución: La API de un LLM no guarda llamadas. Tu aplicación debe reinyectar todo el historial acumulado en cada nueva petición.

03 · ECOSISTEMA DE IA

¿Dónde se sitúa la Memoria conversacional?

Cadenas (Chains)

Secuencian pasos lógicos y estructuran la ejecución paso a paso.

RAG (Conocimiento Estático)

Recupera documentos corporativos o datos externos en tiempo real.

Memoria Conversacional

Sostiene el hilo vivo de la conversación actual y el contexto del usuario.

04 · FLUJO DE DATOS

El Ciclo de Vida de un Mensaje

PASO 01

Petición + ID

Llega el mensaje con su identificador único de sesión.

→

PASO 02

Carga de Estado

El almacén recupera el historial de la sesión.

→

PASO 03

Inferencia LLM

El modelo razona sobre todo el contexto reunido.

→

PASO 04

Guardado

Se añade el nuevo turno al historial de sesión.

05 · INGENIERÍA DE CÓDIGO

Orquestación Práctica en Python

```python
# 1. Almacén de memoria en RAM
from langgraph.checkpoint.memory import InMemorySaver
checkpointer = InMemorySaver()

# 2. Creación del agente
agente = create_agent(model=modelo, checkpointer=checkpointer)

# 3. Invocación con ID de sesión
config = {"configurable": {"thread_id": "sesion-alex"}}
agente.invoke({"messages": [...]}, config=config)
```

InMemorySaver

Guarda el estado del flujo directamente en memoria.

thread_id

Aísla por completo el contexto entre usuarios o chats.

06 · SEGURIDAD Y ALCANCE

Aislamiento por Hilo y Volatilidad

Aislamiento con thread_id

Cada chat tiene su propia clave. Cambiar el ID crea un entorno completamente limpio y aislado sin fugas de datos.

RAM vs Producción

`InMemorySaver` es ideal para pruebas. En entornos de producción, se conecta a Postgres o Redis para no perder estado tras reinicios.

07 · RETOS Y ALTERNATIVAS

Gestión de Tokens y Ventana de Contexto

Coste de Tokens

Enviar historiales gigantes encarece cada llamada exponencialmente.

Degradación del Modelo

Demasiado texto pasado diluye la capacidad de respuesta actual.

Técnicas de Recorte:

- Ventana Deslizante (Últimos N mensajes)
- Resumen Automático
- Búferes FIFO / LlamaIndex

08 · CIERRE Y PRÁCTICA

Las 4 Operaciones Clave: Guardar, Recuperar, Aislar y Borrar.

El éxito de los agentes en producción depende de la precisión de tu arquitectura de estado.

¡Turno para Preguntas!



Image Sources



https://images.presentationgo.com/2025/04/abstract-neural-network-background.png

Source: www.presentationgo.com



https://media.easy-peasy.ai/8f0dd6c2-432a-4079-8cf0-66c2498f9360/8ffc5c95-cf8c-463e-b3dc-7ec7a1b178ce_medium.webp

Source: easy-peasy.ai